

Resultados: comparación delay() contra millis()

Temporización no bloqueante: tres LEDs con delay() contra tres LEDs con millis()

Resultados obtenidos

Parte 1 — delay()

- Los tres LEDs parpadean, pero cada uno enciende una sola vez por ciclo y el ciclo dura 6 s; ninguno cumple su periodo.
- En 6 s se esperaban 6 / 3 / 2 encendidos y se obtuvo 1 / 1 / 1.
- Nunca hay dos LEDs encendidos al mismo tiempo, aunque los periodos pedidos coinciden a los 1500 y 3000 ms.
- Se comprobó que delay() detiene toda la ejecución: mientras un LED "espera", los otros quedan congelados.

Parte 2 — millis()

- Los tres LEDs parpadean a 500 / 1000 / 1500 ms de forma simultánea e independiente, sin ningún delay().
- En 6 s el LED1 enciende 6 veces, el LED2 3 y el LED3 2, tal como se pedía; se observan instantes con dos y tres LEDs encendidos a la vez.
- El reto funciona: el Monitor Serie recibe un mensaje cada 3000 ms exactos y su incorporación no alteró el ritmo de ningún LED.
- El patrón if (ahora - ultimoCambio >= periodo) con unsigned long es seguro frente al desbordamiento de millis().

Comparación de las dos formas de trabajar

Mismo circuito, mismos LEDs, mismos periodos pedidos; solo cambia el programa:

Aspecto	Parte 1 — delay()	Parte 2 — millis()
Ritmo de cada LED	Ninguno cumple su periodo: todos encadenados en un ciclo de 6 s	Cada uno cumple el suyo: 500 / 1000 / 1500 ms
Encendidos en 6 s (LED1 / LED2 / LED3)	1 / 1 / 1	6 / 3 / 2 (lo pedido)
LEDs encendidos a la vez	Nunca más de uno	Hasta los tres (a los 1500 ms)
Duración de una vuelta del loop()	6000 ms	Microsegundos
¿Atiende un botón o el serial mientras "espera"?	No: está bloqueado dentro del delay()	Sí: nunca espera, solo compara
Agregar una tarea nueva (reto: serial cada 3 s)	Hay que reescribir la secuencia y aun así llega tarde	Un renglón más, sin tocar las otras
Uso del procesador	Ocupado "sin hacer nada"	Libre entre comparaciones
Desbordamiento de millis()	No aplica	Resuelto por la resta en unsigned long

Aspecto	Parte 1 — delay()	Parte 2 — millis()
Complejidad del código	Más corto, pero incorrecto	Un struct y un for; correcto y escalable
Dónde sirve	Solo si el programa hace una sola cosa	Cualquier sistema que atienda varias cosas a la vez

En resumen: delay() sirve cuando el programa hace una sola cosa. En cuanto hay dos o más tareas con tiempos distintos, o una entrada que deba atenderse al instante, la única opción correcta es no esperar: guardar cuándo actuó cada tarea y revisar en cada vuelta del loop() si ya le toca.

Tablas de tiempos

Parte 1 — delay() (un ciclo de 6000 ms):

Tiempo (ms)	LED1	LED2	LED3	Instrucción en ejecución
0 – 500	ON	off	off	delay(500)
500 – 1000	off	off	off	delay(500)
1000 – 2000	off	ON	off	delay(1000)
2000 – 3000	off	off	off	delay(1000)
3000 – 4500	off	off	ON	delay(1500)
4500 – 6000	off	off	off	delay(1500)

Parte 2 — millis() (primeros 3000 ms, se repite):

Tiempo (ms)	LED1 (500)	LED2 (1000)	LED3 (1500)	LEDs encendidos
0	off	off	off	0
500	ON	off	off	1
1000	off	ON	off	1
1500	ON	ON	ON	3
2000	off	off	ON	1
2500	ON	off	ON	2
3000	off	ON	off	1

Salida del Monitor Serie de la Parte 2 (9600 baudios)

Parpadeo con millis(): 500 / 1000 / 1500 ms

Mensaje #1 t = 3000 ms LEDs: 010
 Mensaje #2 t = 6000 ms LEDs: 000
 Mensaje #3 t = 9000 ms LEDs: 010
 Mensaje #4 t = 12000 ms LEDs: 000
 Mensaje #5 t = 15000 ms LEDs: 010
 Mensaje #6 t = 18000 ms LEDs: 000

Evidencias

Video Parte 1 (delay): <https://www.youtube.com/shorts/sUPkjt看6g>

Video Parte 2 (millis): https://www.youtube.com/shorts/pJ0OG_M3bCs

Fotos del armado y diagrama de conexión: carpeta Diagrama/ del repositorio.